

Tarea 2

Árboles AVL vs Splay Trees

Profesores: Benjamín Bustos y Gonzalo Navarro

Auxiliares: Claudio Gaete y Gabriel Tapia

1. Indicaciones administrativas

- Fecha de entrega: Lunes 15 de junio. Atrasos hasta el viernes 19 de junio (con descuento).
- Grupos de a lo más 3 personas **de la misma sección**.
- Lenguaje a utilizar: C, C++ o Java.

2. Contexto

El árbol AVL fue presentado en 1962 como el primer árbol de búsqueda binaria autobalanceable, es decir, que está constantemente **equilibrado**. Esto lo consigue mediante una variable adicional denominada *Balance Factor* (o *factor de equilibrio* en español), la cual almacena toda la información necesaria para saber si los hijos de cada nodo están o no balanceados. Lo anterior permite conseguir un costo de $O(\log n)$ para búsqueda, inserción y eliminación tanto promedio como peor caso al mantener una altura máxima de $\approx \frac{3}{2} \log_2 n$ para cada nodo.

El *Splay Tree* es otra estructura de árbol de búsqueda binaria autobalanceable, la cual, sin la necesidad de almacenar información adicional para mantener el equilibrio, alcanza un costo amortizado de $O(\log n)$ para las mismas tres operaciones. Es más, para una secuencia de accesos a nodos con distintas probabilidades tiene un costo amortizado de $O(\mathcal{H})$, donde $\mathcal{H}$ es la entropía de esas probabilidades (profundizaremos en esto más adelante).

El objetivo de esta tarea es comparar ambas estructuras en distintos escenarios de inserción y búsqueda, además de comprobar experimentalmente el desempeño mediante ciertos teoremas probados para los *Splay Trees*.

3. Conceptos preliminares

3.1. Notación

Para esta tarea se ocupará la notación $X = x(A, B)$, para denotar que X es un árbol con el elemento x en la raíz, subárbol izquierdo A y subárbol derecho B . Además, usaremos el símbolo “ $A \rightarrow B$ ” para denotar una rotación de un árbol A a un árbol B .

Por ejemplo:

$$y(x(A, B), C) \rightarrow x(A, y(B, C))$$

representa una rotación simple hacia la derecha, donde x es el hijo izquierdo de y , y luego y pasa a ser el hijo derecho de x .

3.2. Operaciones básicas

Para la implementación de las estructuras, definiremos una serie de rotaciones primitivas, con las cuales podremos realizar operaciones más complejas tales como **splay**:

Rotaciones simples:

- Zig: $y(x(A, B), C) \rightarrow x(A, y(B, C))$
- Zag: $y(A, x(B, C)) \rightarrow x(y(A, B), C)$

Notar que **Zig** corresponde a la rotación simple a la derecha del ejemplo que vimos en el punto anterior.

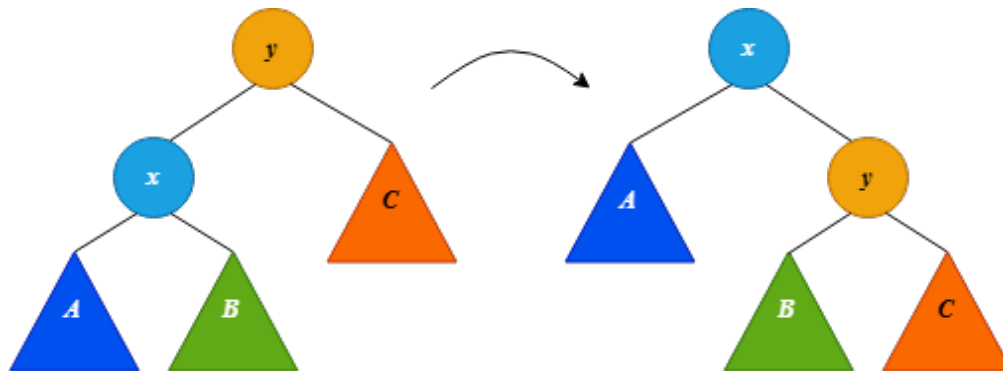


Figura 1: Zig.

Rotaciones dobles:

- Zig-Zig: $z(y(x(A, B), C), D) \rightarrow x(A, y(B, z(C, D)))$
- Zig-Zag: $z(y(A, x(B, C)), D) \rightarrow x(y(A, B), z(C, D))$
- Zag-Zig: $z(A, y(x(B, C), D)) \rightarrow x(z(A, B), y(C, D))$
- Zag-Zag: $z(A, y(B, x(C, D))) \rightarrow x(y(z(A, B), C), D)$

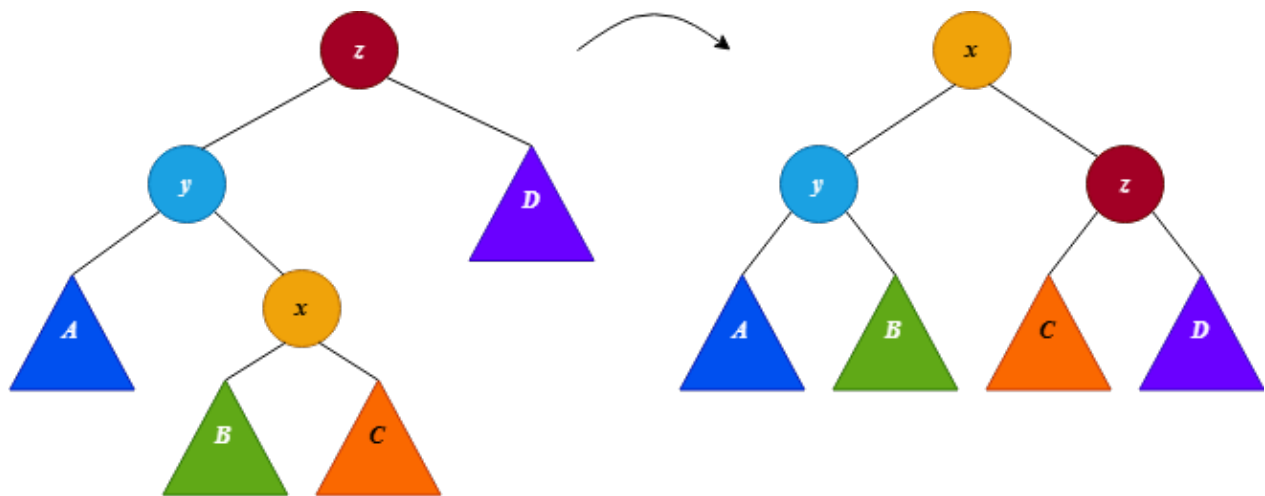


Figura 2: Zig-Zag.

4. Árboles AVL

El *Balance Factor* de un nodo se define como la diferencia entre las alturas del subárbol izquierdo y derecho, es decir:

$$BF(x(A, B)) = height(A) - height(B)$$

Un árbol AVL es árbol de búsqueda binaria donde el factor de equilibrio de cada uno de sus nodos no es mayor a 1 ni menor a -1 , es decir, que su subárbol izquierdo tiene a lo más un nivel más de profundidad que su subárbol derecho (o al revés). De esta forma, se asegura que la altura de cada nodo es menor a $\frac{3}{2} \log_2 n$, como vimos anteriormente. Dicho de otra manera, un árbol AVL es un ABB que mantiene el invariante: $\forall x_i \in x(A, B), |BF(x)| \leq 1$. Diremos que un nodo x está “desbalanceado” si $BF(x) > 1$.

Esta propiedad de los AVL se consigue mediante una rotación luego de cada operación que altere la estructura del árbol (como inserciones o borrados) en caso de ser necesaria.

Importante: Notar que una sola operación no puede aumentar la altura de un nodo en más de una unidad, por lo que simplemente diremos que un nodo x está desbalanceado si $BF(x) = 2$.

4.1. Búsqueda en un AVL

La búsqueda en un árbol AVL se hace de la misma manera que un ABB clásico. Digamos que queremos buscar el elemento x en el árbol $r(A, B)$:

- Si $r = x$, terminamos la búsqueda exitosamente
- Si $x < r$, buscamos x en A recursivamente.
- Si $x > r$, buscamos x en B recursivamente.

Si eventualmente llegamos a un árbol vacío, significa que el elemento x no existe.

4.2. Inserción en un AVL

La inserción en un árbol AVL es similar a la inserción en un ABB clásico, con la diferencia de que debemos verificar el factor de balance en cada llamada recursiva y realizar una rotación en caso de ser necesaria. En efecto, digamos que queremos insertar el elemento x en el árbol $r(A, B)$, describiremos el proceso de inserción a continuación:

- Primero, insertamos x dentro del árbol:
 - Si r es *vacío*, insertamos x en r .
 - Si $x < r$, insertamos en A recursivamente.
 - Si $x > r$, insertamos en B recursivamente.
- Luego, para cada nodo en el camino *de vuelta* a la raíz del árbol:
 - Actualizamos su altura.
 - Recalculamos su factor de equilibrio.
 - Verificamos que siga cumpliendo el invariante: Si no se cumple, debemos balancear con una rotación.

4.3. Rotaciones en un AVL

Como vimos anteriormente, al encontrar un nodo desbalanceado (un nodo z con $|BF(z)| = 2$) en el camino *de vuelta*, debemos realizar una rotación para restaurar el invariante.

En efecto, digamos que z es el nodo desbalanceado, y su hijo de mayor altura y x el nieto que produjo el desbalance. Podemos identificar 4 casos posibles de desbalance:

- **Left Left (LL):** El desbalance está en el subárbol izquierdo del hijo izquierdo, es decir, $z(y(x(A, B), C), D)$. Se identifica si $BF(z) = 2$ y $BF(y) \geq 0$.
 - **Solución:** Se aplica un **Zig** sobre z .
- **Right Right (RR):** El desbalance está en el subárbol derecho del hijo derecho, es decir, $z(A, y(B, x(C, D)))$. Se identifica si $BF(z) = -2$ y $BF(y) \leq 0$.
 - **Solución:** Se aplica un **Zag** sobre z .
- **Left Right (LR):** El desbalance está en el subárbol derecho del hijo izquierdo, es decir, $z(y(A, x(B, C)), D)$. Se identifica si $BF(z) = 2$ y $BF(y) < 0$.
 - **Solución:** Se aplica un **Zig-Zag** sobre z .
- **Right Left (RL):** El desbalance está en el subárbol izquierdo del hijo derecho, es decir, $z(A, y(x(B, C), D))$. Se identifica si $BF(z) = -2$ y $BF(y) > 0$.
 - **Solución:** Se aplica la rotación doble **Zag-Zig** sobre z .

Nota: Como pueden ver, el AVL **nunca** utiliza las rotaciones **Zig-Zig** ni **Zag-Zag** definidas en la sección anterior, a diferencia del *Splay Tree*.

5. Splay Trees

El *Splay Tree* es un tipo de árbol de búsqueda binaria que tiene un método distinto de realizar las operaciones, el cual garantiza un costo amortizado de $O(\mathcal{H})$, donde $\mathcal{H}$ es la entropía de esas probabilidades, la cual es definida como sigue:

$$\text{Dada } \mathbb{P} : \mathcal{U} \rightarrow [0, 1], \mathcal{H}(\mathbb{P}) = \sum_{u \in \mathcal{U}} \mathbb{P}(u) \cdot \log_2 \frac{1}{\mathbb{P}(u)}$$

lo que se traduce a $O(\log n)$ para claves con distribución uniforme (pues $\mathbb{P}(u) = \frac{1}{n}, \forall u \in \mathcal{U}$).

Esto se consigue gracias a que en esta estructura, todas las operaciones (incluso la lectura) modifican la estructura mediante rotaciones, manteniendo el árbol constantemente balanceado al igual que el AVL, pero además mejorando la eficiencia para distribuciones sesgadas de claves gracias a las modificaciones (y sin la necesidad de almacenar información adicional).

La idea principal del Splay Tree es que el nodo más profundo accedido (normalmente el “objetivo” de nuestra operación) debe quedar en la raíz del árbol. Para ello, una vez accedido un nodo x mediante cualquier operación, éste se lleva a la raíz utilizando una operación llamada **splay**(x).

5.1. Splay

Como dijimos anteriormente, luego de acceder a un nodo x , debemos llamar a $\text{splay}(x)$ para subirlo hasta la raíz. Esta operación consiste de una serie de rotaciones consecutivas (como las descritas en la [sección 3.2](#)), la cual finalizará una vez que x esté en la raíz del árbol.

Es importante mencionar que las rotaciones simples (**zig** y **zag**) solo se realizarán si el nodo y es la raíz del árbol, por lo que la secuencia de rotaciones llamadas por **splay** consistirá de 0 o más rotaciones dobles consecutivas seguidas de una posible rotación simple.

5.2. Búsqueda en un Splay Tree

La búsqueda en un Splay Tree se hace de la misma manera que en un ABB clásico, o, lo que es lo mismo, de la misma forma que un AVL (ver [sección 4.1](#)). Luego, se hace $\text{splay}(x)$.

Si x no se encuentra, se hace $\text{splay}(x')$, donde x' es el último nodo visitado, o mejor dicho, el nodo que sería el padre de x en caso de estar este último en el árbol.

5.3. Inserción en un Splay Tree

La inserción en un Splay Tree se hace de la misma manera que en un ABB clásico. En efecto, digamos que queremos insertar el nodo x en el árbol $r(A, B)$:

- Si $r = \text{none}$, se crea un nodo $x(\text{none}, \text{none})$ y se inserta en lugar de r .
- Si $x \leq r$, insertamos en A recursivamente.
- Si $x > r$, insertamos en B recursivamente.

Luego de finalizada la inserción del elemento, se hace $\text{splay}(x)$.

5.4. Sequential Access Theorem

El *Static Finger Theorem* establece que dado un elemento fijo f del universo, llamado “dedo” (*finger*), el costo total de una secuencia de m accesos $x_1, x_2, \dots, x_m$ en un Splay Tree con n elementos es:

$$O\left(n \log n + m + \sum_{i=1}^m \log(|x_i - f| + 1)\right)$$

Es decir, el costo de acceder a un elemento x_i es proporcional al logaritmo de su distancia al dedo f : cuanto más cercano a f es el elemento accedido, menor es su costo.

Una consecuencia directa de este resultado es el **Sequential Access Theorem**: si se buscan m claves en orden estrictamente creciente en un Splay Tree con n elementos, entonces el costo total de la secuencia es:

$$O\left(n + m \log\left(\frac{n}{m}\right)\right)$$

5.5. Working Set Theorem

El *Working Set Theorem* establece que el costo de acceder al elemento x en el instante t es $O(\log(t(x) + 1))$, donde $t(x)$ denota el número de elementos *distintos* accedidos entre la última vez que se accedió a x y el instante actual. A este valor se le denomina el *tamaño del working set* de x en el instante t .

Intuitivamente, los elementos accedidos recientemente ascienden hacia la raíz y son baratos de volver a acceder. Más formalmente, si una aplicación trabaja repetidamente sobre un subconjunto activo de W elementos (el *working set*), el costo amortizado por acceso en un Splay Tree de n nodos será $O(\log W)$ en lugar de $O(\log n)$.

6. Implementación

6.1. Objetivos

Se espera que implementen ambas estructuras de datos junto con sus operaciones fundamentales. Es decir, para cada estructura, deben implementar al menos los siguientes métodos:

- **insert(x):** Inserta un nuevo elemento con clave x en el árbol.
- **search(x):** Busca el elemento x en el árbol. Puede retornar un booleano o un puntero al nodo, pero lo fundamental es que ejecute toda la lógica de recorrido y reestructuración correspondiente.

Adicionalmente, se les exigirá contar con funciones de rotación que representen las primitivas descritas en la [sección 3.2](#), con las cuales deberán construir las operaciones más complejas como **splay(x)** y el balanceo del árbol AVL si así lo desea (esto es bastante recomendado).

7. Experimentación

7.1. Construcción

7.1.1. Creación de datasets

Para cada $N \in \{2^{10}, 2^{11}, \dots, 2^{14}\}$, deberán construir un dataset de números aleatorios uniformemente distribuidos en el rango $[0, 4294967295 = 2^{32} - 1]$. Estos pueden ser guardados en un arreglo, en un archivo de texto plano o en la forma que estimen conveniente.

7.1.2. Generación de función aleatoria

Deberán definir una función de probabilidad sesgada basada en la distribución exponencial:

$$\mathbb{P}(i) = e^{-\lambda i} \cdot \frac{1 - e^{-\lambda}}{1 - e^{-\lambda N}}$$

donde tendrán que fijar el parámetro λ para crear su propia función sesgada: Valores cercanos a 0 generan distribuciones casi uniformes y cercanos a 1 generan distribuciones demasiado sesgadas (recomendamos $\lambda \in [0.001, 0.05]$ para obtener resultados interesantes).

7.2. Escenarios Base

Para comenzar, deberán fijar un $c \in \{1, 2, \dots, 10\}$ a elección (deben incluir este valor en su código y en su informe). Luego, para cada $N \in \{2^{10}, 2^{11}, \dots, 2^{14}\}$, deberán realizar N inserciones en ambas estructuras, seguidas de $M = 10c \cdot N$ búsquedas para 4 configuraciones experimentales definidas más adelante. Los elementos escogidos tanto para la inserción como para la búsqueda deben ser obtenidos del dataset asociado al N actual.

Además, tendrán que reportar los tiempos totales tanto de inserción como de búsqueda para todos los casos en una tabla y realizar un gráfico de línea para las 4 configuraciones que permita comparar los tiempos de ejecución de ambas estructuras para cada valor de N , el cual muestre el tiempo para cada búsqueda de modo de exponer visualmente la naturaleza amortizada del Splay Tree.

Las configuraciones son descritas a continuación:

- Inserción aleatoria, búsqueda uniforme:** Realicen N inserciones eligiendo elementos aleatorios con probabilidad uniforme. Luego, realicen M búsquedas sobre elementos aleatorios escogidos con probabilidad uniforme.
- Inserción aleatoria, búsqueda sesgada:** Realicen N inserciones eligiendo elementos aleatorios con probabilidad uniforme. Luego, realicen M búsquedas sobre elementos aleatorios escogidos utilizando su función $\mathbb{P}(i)$ definida anteriormente.
- Inserción ordenada, búsqueda uniforme:** Antes de insertar, ordenen los elementos de menor a mayor y realicen N inserciones secuenciales. Luego, realicen M búsquedas sobre elementos aleatorios con probabilidad uniforme.
- Inserción ordenada, búsqueda sesgada:** Antes de insertar, ordenen los elementos de menor a mayor y realicen N inserciones secuenciales. Luego, realicen M búsquedas sobre elementos aleatorios escogidos utilizando su función $\mathbb{P}(i)$ definida anteriormente.

Entregue una breve conclusión sobre los resultados obtenidos, comparándolos con las cotas conocidas para ambas estructuras.

7.3. Teoremas y Naturaleza de los Splay Trees

Además de los escenarios base, deberán comprobar el comportamiento de ambas estructuras frente a los dos teoremas enunciados anteriormente para un nuevo dataset $\mathcal{N}$ con $N = 2^{25}$ elementos aleatorios en el rango $[0, 4294967295 = 2^{32} - 1]$ generado por ustedes.

Luego de eso, deberán crear tanto un AVL tree como un Splay Tree para ambos experimentos, cuidando que los elementos hayan sido insertados en el mismo orden para cada uno de los cuatro experimentos de esta sección.

Finalmente, deberán mostrar el desempeño de ambas estructuras frente a los dos escenarios experimentales descritos a continuación en un gráfico de línea.

7.3.1. Experimentos

- a) **Sequential Access Theorem:** Para los valores de $m \in \{\frac{N}{100}, \frac{2N}{100}, \dots, \frac{9N}{100}, \frac{N}{10}\}$, ejecuten una secuencia de m búsquedas de claves **estrictamente crecientes** no necesariamente equiespaciadas y grafiquen el tiempo de ejecución para cada valor de m en ambas estructuras. Pueden controlar el crecimiento con algún parámetro o función a elección, pero se recomienda recorrer las claves aumentando su valor en un número fijo.
- b) **Working Set Theorem:** Para $W \in \{10, 10^2, 10^3, \dots, 10^6\}$ generen un subconjunto $\mathcal{W} \subseteq \mathcal{N}$ aleatorio de tamaño W (el *working set*) y realicen una secuencia de $M = 10c \cdot N$ búsquedas de elementos escogidos uniformemente al azar exclusivamente dentro de dicho subconjunto en ambos árboles y grafiquen el tiempo de ejecución para cada valor de W .

7.3.2. Análisis de resultados

Para las secuencias de búsquedas del *sequential access theorem*, comprueben que en el árbol AVL toman un tiempo de $O(m \log n)$, mientras que el Splay Tree es capaz de aprovechar esta secuencia y resolverlas en $O(m \log(\frac{n}{m}))$.

Para las secuencias de búsquedas del *working set theorem*, comprueben que el costo escala como $O(\log W)$ en el Splay Tree, mientras que en el árbol AVL permanece como $O(\log n)$.

7.4. Bonus (+1.0 pts): Traversal Conjecture

Así como existen muchos teoremas probados acerca del desempeño de los Splay Trees bajo ciertas condiciones, también quedan bastantes conjeturas sin probar (inclusive algunas se remontan a la presentación original de los Splay Trees en 1985). En particular, en esta sección trataremos la *Traversal Conjecture*.

7.4.1. Enunciado

La *Traversal Conjecture* postula que dados T_1, T_2 dos Splay Trees con los mismos elementos y dada S la secuencia obtenida producto de visitar todos los nodos de T_2 en preorden, el costo total de buscar secuencialmente los elementos de S en T_1 es $O(n)$.

7.4.2. Experimento

Construyan dos Splay Trees T_1 y T_2 de tamaño $N = 2^{25}$ cada uno, insertando los mismos elementos pero con distintas permutaciones (pueden reutilizar el dataset $\mathcal{N}$). Luego, ejecuten un recorrido completo en algún orden a elección (preorden, inorden o postorden) sobre T_1 , para luego buscar esa misma secuencia en T_2 . Entreguen el costo total de este proceso, muestren los tiempos de cada búsqueda en un gráfico de línea y emitan una conclusión evaluando empíricamente la conjetura con respecto a los resultados obtenidos.

8. Entregables

Se deberá entregar el código fuente y un informe en formato PDF con los resultados y conclusiones obtenidas luego de la experimentación. La nota final de la tarea (NT_2) será el promedio simple entre la nota de código ($NCod$) y la nota de informe ($NInf$):

$$NT_2 = \frac{1}{2} \cdot NCod + \frac{1}{2} \cdot NInf$$

8.1. Código

La entrega de código tiene contener:

- **(0.3 pts) README:** Archivo con las instrucciones claras para compilar y ejecutar el código. Debe ser lo suficientemente explicativo para que cualquier persona pueda ejecutar la totalidad de sus experimentos.
- **(0.2 pts) Firmas:** Cada estructura de datos y función relevante debe tener una descripción de su propósito, parámetros de entrada y salida.
- **(1.0 pts) Implementación AVL:** Inserción, búsqueda y rotaciones correctas. Preserva el invariante descrito.
- **(2.0 pts) Implementación Splay Tree:** Inserción, búsqueda, rotaciones y el método `splay` son correctas.
- **(1.5 pts) Experimentos:** Creación correcta de los 4 escenarios base y los 2 generadores de secuencias para los teoremas.
- **(0.5 pts) Obtención de resultados:** La forma en la que se miden los tiempos es rigurosa y suficiente para emitir conclusiones.
- **(0.5 pts) Main:** Un archivo principal o parte del código (una función `main`) que permita ejecutar toda la batería de tests sin modificar el código.
- **(1.0 pts extra) Bonus:** Implementación, testeo y conclusión correspondiente al experimento.

8.2. Informe

El informe debe ser claro, analítico y conciso. Se recomienda encarecidamente utilizar LaTeX o Typst. Debe contener:

- **(0.8 pts) Introducción:** Presentación del tema en estudio y resumen de lo que se abordará en el informe.
- **(0.8 pts) Desarrollo:** Breve explicación de las estructuras y cómo funcionan. Dado que el cuerpo docente ya conoce la teoría, enfóquense en explicar **sus** decisiones de implementación. En esta sección se espera que incluyan las constantes y funciones determinadas para su implementación.

- **(2.4 pts) Resultados:** Especificación detallada del *setup* experimental (hardware, SO, caché, RAM de la computadora donde se ejecutaron los experimentos). Se deben mostrar los gráficos y tablas solicitadas, con sus ejes correctamente etiquetados.
- **(1.2 pts) Análisis:** Discusión de los resultados obtenidos. En esta sección se espera que respondan las preguntas de análisis que se fueron realizando a lo largo de la tarea.
- **(0.8 pts) Conclusión:** Recapitulación del trabajo. Conclusión final sobre el desempeño de ambas estructuras.

9. Recomendaciones para la tarea

- a) Puede ser buena idea programar los métodos de rotación primero en los Splay Trees de modo que puedan ser replicados para los árboles AVL y así ahorrarse trabajo.
- b) En sus gráficos de línea incluyan las funciones de las cotas teóricas multiplicadas por alguna constante a determinar que les permita comparar más simplemente los resultados obtenidos.
- c) Mucho cuidado con la notación de los árboles AVL: Recuerden que $BF > 0$ significa un desbalance hacia la izquierda, mientras que $BF < 0$ significa un desbalance hacia la derecha. Mencionamos esto pues podría ser un poco contrario a lo que intuitivamente podrían llegar a pensar.
- d) Notar que los experimentos de la [sección 7.3](#) requieren la construcción de 4 árboles de tamaño $\approx 2^{25}$ nodos, lo que dependiendo de su implementación, podría traducirse en aproximadamente 4 GB de RAM. Recomendamos fuertemente tener esto en cuenta a la hora de elegir el equipo en el cual ejecutar los experimentos pedidos.
- e) Por favor no incluyan los datasets generados en sus entregas 🙏🙏🙏.
- f) Pueden encontrar la publicación original en el [siguiente enlace](#).